USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Locking Primitives

UM-NATOS-014

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-014	1.2 · 2026-08-15	**Complete, verified on hardware** — §10 added, the same defect from the other side	Internal

1. Abstract

nat-os had no way to make anything mutually exclusive. UM-NATOS-010 §8 recorded the heap as task-context-only with no locking; UM-NATOS-013 §7 recorded console output interleaving as the first thing that would need a lock the kernel did not have. Both are now closed.

Two primitives were built, not one, because the two motivating problems have opposite shapes: the heap needs a handful of memory operations to be indivisible, and the console needs a 13-millisecond write to be indivisible. Solving both with the same tool would mean either an unsafe heap or a wrecked scheduler.

The blocking mutex needed three attempts. The second was correct and starved a task to a standstill; §5 is that measurement.

Revision 1.2 adds §10. The batching fix in §5.2 held, and the same defect returned from the opposite direction once applications began drawing: they took the lock per primitive against a renderer holding it per frame. Measuring it required distinguishing **blocked** time from **lock-busy** time, which are not the same and differed here by a factor of three — the lock was free 88% of the time while two tasks were blocked on it.

2. Two primitives

	Critical section	Blocking mutex
Mechanism	Raise `PS.INTLEVEL` to mask the tick	Block the task, remove it from the rotation
Cost while held	Scheduling latency, for the whole duration	Two context switches per handoff
Correct for	A few memory operations	Anything longer
Usable from an ISR	Yes	**No** — a handler cannot block
Used by	`heap_alloc`, `heap_free`	Console, application-shared state

The critical section is the wrong tool for the console. Writing a 150-character line at 115200 baud takes about 13 ms. Masking the tick for that long to solve a cosmetic interleaving problem would degrade every scheduling deadline in the system. That asymmetry is why both exist.

`crit_enter()` returns the previous interrupt level and `crit_exit()` restores it, rather than forcing 0. Nesting therefore works, and a section inside an interrupt handler cannot silently re-enable interrupts on the way out.

3. Blocking, and what it required

A mutex that blocks needs the scheduler to understand "not runnable", which it did not. Three changes:

- `TASK_BLOCKED`**.** The scheduler skips it. `TASK_READY` was previously the only live state (UM-NATOS-010 §8).
- `task_block()` **does not yield.** The caller marks itself blocked while holding a critical section — so the decision to block and the record of what it waits for cannot be split by a tick — then leaves the section and yields. That split is the classic lost wakeup: the holder releases, sees an empty wait list, and the waiter sleeps forever.
- **An idle task.** With blocking, a moment where every task is waiting has nothing to switch to, and the old fallback of resuming the interrupted task would run a task that had just blocked. Idle is registered via `task_set_idle()` and kept **out of the round robin**, chosen only when nothing else can run — leaving it in the rotation would spend a seventh of the machine deliberately doing nothing. It executes `WAITI 0`, which stops the core until an interrupt arrives.

4. Ownership sentinel

`MUTEX_FREE` is `-2`, not `-1`. Before the scheduler starts, `task_current()` returns `-1`, so using `-1` for "unheld" would make an unowned mutex indistinguishable from one held by the boot context, and the free and recursive tests would both match. A one-line problem that would have produced a mutex quietly granting itself to everyone during startup.

The mutex is **recursive**. A console lock is exactly the kind of thing that acquires a nested acquisition by accident, and self-deadlock on a kernel with no debugger is an expensive way to discover it.

5. The starvation defect

The first working mutex released the lock and woke every waiter to race for it. The reasoning recorded in the source was that with at most eight tasks a thundering herd was not worth managing.

That reasoning was wrong, and the instrumentation said so precisely:

```
lock owner=1  waiters=0x00000004  acq=238542  contended=137  err=0  states=1121111
work a/b=238541/0
```

Worker-a had acquired the lock **238,542 times**. Worker-b — `waiters` bit 2, state 2, `TASK_BLOCKED` — had acquired it **zero times**, and had not completed a single iteration of its loop.

This was not a lost wakeup. Worker-b was being woken correctly and re-blocking every time. Worker-a's loop is tight enough that it re-acquired the lock long before the woken waiter was next scheduled. **Race-on-release provides no ordering guarantee at all**, and against a tight loop that is not merely unfair — it starves the waiter indefinitely.

5.1 Fix — direct handoff

`mutex_unlock()` now transfers ownership to a waiter rather than freeing the lock. The successor is the owner *before* it runs again, so nothing can cut in front.

This reintroduces the complication the first design was avoiding: `owner` names a task that is not yet on the CPU. A `granted` bitmask resolves it. A task woken by handoff sees its bit set and returns holding the lock, and that test is made **first** — before the `owner == me` test, which would otherwise read a handoff as a recursive acquisition and leave `depth` permanently too high.

5.2 The cost of fairness

With the handoff in place and the lock taken on *every* worker iteration:

```
work a/b=2062/2061      (was 238541/0)
```

Fair, but throughput collapsed roughly thirtyfold. A handoff costs a full scheduling round-trip — the successor cannot run until the next tick — so a lock contended on every iteration becomes bounded by the tick rate rather than by the work.

That is a real property of a blocking lock, not a defect, but it is a pathological workload. Contending once every 32 iterations is representative, and restores throughput:

```
work a/b=50143/50143    acq=3133  contended=200  err=0  skew=0
```

Both workers now advance **exactly** in step. The lesson worth keeping is that a blocking mutex is the wrong instrument for a lock contended at high frequency; that case wants a critical section, or a design that does not share.

6. Results

```
[6a] critical : PASS  ticks held at 1 across 2 periods, resumed at 7
[6b] mutex    : PASS  recursive depth, ownership, non-owner unlock refused (1),
                      try_lock both ways

t=809  work a/b=50143/50143  guards=ok  corrupt=0
       lock owner=1 waiters=0x00000004 acq=3133 contended=200 err=0 skew=0
       states=1121111
```

6.1 Critical sections mask the tick

Held across two full tick periods with the tick count frozen at 1, then resumed at 7 once the level dropped. Both halves matter: freezing proves interrupts were masked, and resuming proves the tick was **deferred rather than lost**. Masking that silently dropped ticks would keep the scheduler running while making every timeout wrong.

This test originally ran in `m6_selftest()` before `timer_start()`, where `timer_ticks()` is 0 and stays 0 regardless of what masking does — a test that could only ever pass by accident. It reported FAIL, which is the one outcome that made the flaw visible. It now runs from the reporter task with the tick live.

6.2 Mutex semantics

Deterministic, single-threaded: unheld at init, ownership recorded on acquire, recursive acquisition raising depth to 2 without deadlock, depth unwinding correctly, release returning to `MUTEX_FREE`, unlock by a non-owner **refused and counted** rather than acted on, and `try_lock` succeeding when free and failing when held.

Refusing a non-owner unlock matters more than it looks: clearing another task's ownership would admit two holders to the protected section, corrupting whatever it guards far from where the mistake was made.

6.3 Mutual exclusion under contention

`skew=0` throughout. Each worker increments a shared counter under the lock through a deliberately non-atomic read-modify-write with a widened window; the kernel checks that counter against the workers' own independently maintained tallies. Unprotected, this drifts by thousands within a second.

`err=0` — no non-owner unlocks. Task states `1121111` show one task blocked and the rest ready, alternating as the lock changes hands.

6.4 Console arbitration

`ps` output arrived as four unbroken lines with no report interleaved, where previously a report would land mid-table.

Locking is at **message granularity**, not per character. `uart_putc()` stays lock-free, and the panic handler ignores this layer entirely — it runs after a fault, possibly with the lock held by the task that faulted, and a panic that deadlocks trying to report a panic is the worst failure mode available to a kernel with no debugger.

6.5 Regression

M3, M4 and M5 self-tests all still passing. Stack guards intact, `corrupt=0`, headroom 449 words.

7. Metrics

Quantity	Value
Primitives	2
Critical section cost	`RSR` / `WSR` on `PS`, no memory
Mutex size	24 B
Tasks	7 (6 working + idle)
Lock acquisitions measured	3,133
Contentions	200
Non-owner unlocks	0
Lost updates	0
Throughput, pathological contention	~2,060 iterations
Throughput, 1-in-32 contention	~50,143 iterations
Image size	16,256 B
Draw lock held, applications running	12% of wall clock
Blocked per contention	63 ms, against a 24 ms hold
Effect of narrowing the hold 25%	none
Frames/s, blocking → best-effort drawing	3.0 → 9.9
Primitives skipped under best-effort	3.4%

8. What this does not establish

- ~~No priority inheritance.~~ Superseded by §9. What remains is that a boost is dropped on the first release, so nested holds of two boosted mutexes lose it early.
- **No deadlock detection.** Two tasks taking two mutexes in opposite orders will hang, and the kernel will not say so. The idle task will simply run.
- **No timeouts.** `mutex_lock()` waits forever.
- **No condition variables or semaphores.** There is no way to wait for an event, only for a lock.
- **Keystroke echo is not arbitrated.** The shell echoes characters outside the lock, so a report can still split a partially typed line. Locking each keystroke would serialise the console against a human typing speed.
- **Critical sections are unbounded by convention only.** Nothing measures or enforces how long one is held; a long one silently degrades scheduling.
- **Untested against an ISR.** The mutex is documented as unusable from an interrupt handler and nothing enforces that.

- **Best-effort drawing has no fairness of its own.** A skipped primitive is simply lost; nothing retries it, and nothing guarantees an application gets the panel eventually. At 3.4% that is invisible, but the policy contains no bound on how unlucky one application can be.
- **The skip ratio is measured under one workload.** Four small test programs drawing into strips. An application doing sustained full-viewport blits would contend far harder, and nothing here says what its skip rate would be.
- `try_lock` **does not compose with batching.** An application wanting several primitives drawn atomically has no way to ask for that, and would see them skipped independently.

9. Priority inheritance

Revision 1.1. §8 recorded that there was no priority inheritance, on the grounds that without priorities there could be no inversion. Priorities now exist (UM-NATOS-009 §11), so the hazard became real the same day.

9.1 Why it was needed immediately

The renderer runs at HIGH and takes the display lock. The applications beneath it run at NORMAL and take the same lock. A NORMAL task holding it while the renderer waits can be preempted by any other NORMAL task, so the renderer waits not for the critical section but for the whole NORMAL band to drain.

That is classical priority inversion, and it appeared the moment there was anything to invert.

9.2 What it does

A task blocking on a held mutex raises the owner to its own priority, bounding the wait to the length of the critical section rather than to the scheduler's behaviour. The boost is dropped when the lock is released or handed on.

Release restores the **base** priority rather than tracking nested holds, so a task holding two boosted mutexes loses the boost on the first release. Recorded rather than solved: nothing in this kernel holds two, and inventing a nesting scheme with no consumer would be guessing at requirements.

9.3 Frequency still matters more

Inheritance bounds the *cost of waiting*. It does nothing about §5.2's lesson, which is about the *number of waits* — and that lesson was rediscovered afterwards by walking into it. A raycaster took the display lock once per column and spent 25 seconds per frame on 37 ms of work; holding it once per frame instead was a **194×** improvement.

Inheritance would not have helped there at all. The fix for a lock contended at high frequency is to contend less often, and `display_lock()` / `display_unlock()` now exist so a caller can hold across a batch.

10. The same defect, from the other side

Added in revision 1.2, after the renderer was measured at 3 frames a second.

§5.2 established that a blocking mutex is the wrong instrument for a lock contended at high frequency, and fixed the raycaster by batching: one acquisition per frame instead of 240. That fix held. What it did not anticipate is that the **other** party would arrive later and reintroduce the same shape from the opposite direction.

Applications draw through `vm.c`, which took the draw lock **per primitive** — exactly what the raycaster used to do to itself. The renderer holds the lock for a whole frame; the applications interrupt it constantly with short takes.

10.1 Blocked time is not lock-busy time

The mutex counted acquisitions and contentions from the beginning. Neither is a duration, and the question here was a duration, so `display.c` was instrumented to time both the hold and the interval between asking for the lock and getting it. Measured over 8.08 s with applications running:

quantity	value
lock held	979 ms — **12% of wall clock**
aggregate blocked	13,051 ms — 1.6 tasks blocked at all times
acquisitions	100
contentions	202 — two blocked waiters per acquisition

The lock was free 88% of the time, and thirteen seconds of blocking accumulated in eight seconds of wall clock. That is only possible because "blocked" is not "lock busy": a task that cannot have the lock is descheduled, and the interval includes being selected again afterwards. 13,051 ms across 202 contentions is **63 ms per contention against a 24 ms hold** — most of it spent getting back onto the CPU.

The accessors are named `display_lock_blocked_*` for this reason. Calling them `wait` would assert the panel was busy for 63 ms, which is false, and would point the next person at hold duration.

10.2 The obvious fix, and why it failed

Which is exactly where it pointed this one. Composition writes to a private buffer and touches no shared hardware, so the lock was narrowed to cover only the blit:

	before	after
lock held	979 ms	**734 ms** (−25%, as designed)
blocked	13,051 ms	**13,085 ms** (unchanged)
frames/s	3.0	3.2

The change did precisely what it was built to do and moved the outcome by nothing. **Hold duration was never the cost.** The narrowed hold was kept anyway, because a lock should cover the shared thing rather than the whole operation that happens to use it — but it is not an optimisation and is not recorded as one.

10.3 Attribution decided the fix

The aggregate says the system is blocked and cannot say on whom, and the two readings imply opposite remedies: making application draws non-blocking helps only if applications are waiting; shortening application holds helps only if the renderer is. Per-task attribution answered it:

```
blocked, display task    3,880 ms    65% of wall clock
blocked, application host 5,894 ms    98% of wall clock
```

Both, mutually — contending not for the panel, which was idle, but for the CPU afterwards.

10.4 Stop blocking

`vp_fill`, `vp_text` and `SYS BLIT` now take the lock with `display_try_lock()` and skip the primitive when it is busy. A dropped fill costs one frame of one application; a blocked task costs the whole system a scheduling round-trip.

	before	after
frames/s	3.0	**9.9**
blocked, applications	5,894 ms	**0**
contentions	202 per 8 s	10 total
primitives skipped	—	45 of 1,325 = **3.4%**

Within 11% of the 11.1 fps measured with every application killed, so nearly all the lost frame rate is recovered with the applications still running.

Skips are counted rather than hidden. A best-effort policy that silently discards work is indistinguishable from a broken one, and the ratio is the only thing that distinguishes "reasonable" from "starving applications of the screen".

10.5 What §5.2 got right, and what it left out

§5.2's conclusion — "*a blocking mutex is the wrong instrument for a lock contended at high frequency*" — is correct and this is another instance of it. What it left implicit is **why frequency and not duration**, and that turns out to be the actionable half:

> The cost of a contended blocking lock is dominated by the **number of blocking events**, not the time the lock is held. Each one costs a scheduling round-trip whether the lock was held for 24 ms or 24 µs. So the levers are batching (fewer takes) or not blocking at all (`try_lock`) — and shortening holds, the intuitive move, does nothing.

§5.2 reached for the first lever. §10.4 reaches for the second, because the renderer and the applications cannot be batched together: they are different tasks with no common boundary to batch across.

11. References

- UM-NATOS-010 §8 — the heap's task-context-only limitation, closed here
- UM-NATOS-013 §7 — console interleaving, closed here
- UM-NATOS-009 — the scheduler that `TASK_BLOCKED` extends
- `kernel/critical.h` — masking, and why it is wrong for long sections
- `kernel/mutex.c` — handoff, the `granted` bitmask, and §5
- `kernel/console.c` — message-granularity arbitration